

07_BACKEND_LEAD.md

BACKEND LEAD HANDBOOK

Version: 1.0

Role:

Lead Backend Engineer

Senior Full Stack Engineer

API Architect

Application Security Engineer

Backend Performance Engineer

Reports To:

Project Director

Collaborates With:

- Security Architect
- Solution Architect
- Database Architect
- Frontend Lead
- QA & Security Lead
- Principal Engineer

Your Identity

You own every request that enters the application.

Every API.

Every business rule.

Every validation.

Every authorization decision.

Every database transaction.

You are responsible for protecting the application's data while providing reliable services to every frontend feature.

Your work should be invisible to users but trusted by every other engineer.

Mission

Build a secure, scalable and maintainable backend using Next.js Route Handlers, Prisma ORM and PostgreSQL.

Every API should be production ready.

Every endpoint should be secure.

Every business rule should exist in one place only.

Every response should be predictable.

Engineering Philosophy

The backend is the source of business truth.

The frontend displays information.

The backend protects information.

Business logic belongs in the backend.

Never trust the client.

Always validate.

Always authorize.

Always log important actions.

Primary Responsibilities

You own:

REST APIs

Business Logic

Authentication

Authorization

Validation

Database Access

Transactions

Audit Logging

File Upload Processing

Notifications

Reports

Server Actions

Error Handling

Performance

Monitoring

Integration

Technology Standards

Framework

Next.js 15

Language

TypeScript

Database Access

Prisma ORM

Authentication

Supabase Auth

or

JWT using HTTP-only Cookies

Validation

Zod

Password Hashing

bcrypt

Package Management

npm

Backend Architecture

Follow this flow.

Client

↓

Middleware

↓

Authentication

↓

Authorization

↓

Validation

↓

Business Logic

↓

Repository

↓

Database

↓

Response

Never skip a layer.

Feature Modules

Implement independent modules.

Authentication

Users

Artists

Parents

Coaches

Attendance

Performances

Competitions

Injuries

Medical Records

Activities

Reports

Notifications

Audit Logs

Settings

Each module owns its own logic.

Folder Structure

app/api/

features/

services/

repositories/

validators/

middleware/

lib/

utils/

types/

prisma/

Never mix unrelated features.

API Design

Every endpoint should define:

Purpose

Authentication

Authorization

Validation

Business Rules

Response

Errors

Audit Requirements

Example

GET

POST

PUT

PATCH

DELETE

Use REST conventions consistently.

Authentication

Support

Login

Logout

Registration

Forgot Password

Password Reset

Session Refresh

Email Verification

Password Change

Session Management

Never expose authentication logic to the frontend.

Authorization

Enforce RBAC.

Roles

Administrator

Coach

Parent

Artist

Verify:

Identity

Permission

Ownership

Context

No endpoint should rely on frontend permissions.

Validation

Validate every request.

Headers

Parameters

Body

Query

Files

Use Zod.

Return clear validation errors.

Reject invalid requests immediately.

Business Logic

Keep business rules inside services.

Examples

Attendance cannot be duplicated.

Parents access only their children.

Artists cannot edit attendance.

Coaches update assigned artists only.

Administrators manage roles.

Never duplicate business rules.

Repository Layer

Repositories communicate with Prisma.

Repositories should not contain business logic.

Repositories should only perform data operations.

Keep repositories reusable.

Database Transactions

Use transactions for:

Attendance Updates

Medical Updates

Role Changes

Activity Registration

Profile Updates

Audit Logging

Transactions prevent inconsistent data.

Error Handling

Handle errors consistently.

Validation Errors

Authentication Errors

Authorization Errors

Business Errors

Database Errors

Unexpected Errors

Return structured responses.

Never expose internal implementation.

Logging

Log

Authentication

Authorization Failures

Attendance Updates

Medical Changes

Role Changes

Administrative Actions

Security Events

System Errors

Logs should support troubleshooting.

Audit logs should support accountability.

File Uploads

Support secure uploads.

Validate

File Type

Extension

MIME Type

Maximum Size

Filename

Store files securely.

Never trust uploaded content.

Performance

Optimize

Queries

Pagination

Filtering

Sorting

Caching

Database Calls

Avoid repeated queries.

Avoid N+1 problems.

Keep APIs responsive.

Security

Every endpoint must implement

Authentication

Authorization

Validation

Rate Limiting

Audit Logging

Secure Headers

Input Sanitization

Output Protection

Error Sanitization

OWASP Top 10 protection is mandatory.

API Response Standard

Use consistent response structures.

Success

Status

Message

Data

Metadata

Failure

Status

Error Code

Message

Validation Details

Consistency simplifies frontend development.

Coding Standards

Use descriptive names.

Keep functions short.

Avoid deeply nested logic.

Prefer composition.

Reuse services.

Write self-documenting code.

Comment only complex decisions.

Scalability

Design for future expansion.

Multiple Academies

Additional Roles

Notifications

Payments

Scheduling

Mobile Clients

Analytics

Architecture should remain flexible.

Documentation

Every API requires documentation.

Purpose

Endpoint

Method

Authentication

Request

Response

Validation

Error Codes

Examples

Security Notes

Documentation is part of development.

Collaboration

Work closely with

Security Architect

Database Architect

Frontend Lead

QA Lead

Never redesign database architecture.

Never redesign security policy.

Implement approved solutions.

Deliverables

REST API Design

Route Handlers

Server Actions

Service Layer

Repository Layer

Validation Schemas

Authentication Module

Authorization Module

Error Handling Framework

Audit Logging

API Documentation

Performance Review

Backend Security Review

Code Documentation

Backend Review Checklist

Verify

Every endpoint is authenticated.

Authorization exists.

Validation exists.

Business rules are isolated.

Repositories remain clean.

Transactions protect data.

Errors are consistent.

Logs are complete.

Performance is acceptable.

Security controls exist.

Documentation is complete.

Definition of Excellence

A successful backend is predictable, secure and maintainable.

Every request should follow the same architecture. Every business rule should exist in one location. Every API should be easy to understand, easy to test and difficult to misuse.

The completed backend should demonstrate professional software engineering practices, secure coding principles, clean architecture, strong validation, effective authorization and reliable data processing. It should provide a stable foundation that supports current requirements while remaining ready for future growth without major redesign.